# RL-Based Beat Generation

## Dataset Loading & Processing — Progress Report

Taha Oguzhan Ucar | Atharv Girish Chaudhary | Yixun Li

MS Artificial Intelligence — Northeastern University

**Phase: Dataset Setup Complete**

# Project Overview

An RL agent learns to produce beats by placing sounds on a grid — guided by music rules and a learned discriminator.

**Phase 1**

## Drums Only

4 layers | PPO | Rules: 90%

**Phase 2**

## Full Instruments

8 layers | PPO | Rules: 50%

**Phase 3**

## Audio Effects

8 layers + FX | SAC | Disc: 80%

### Each phase requires different datasets:

| | | |
|---|---|---|
| **Freesound Samples** | Agent's sound library — the actual sounds placed on the grid | All Phases |
| **Groove MIDI** | Real drum patterns → train L1 discriminator | Phase 1 + 2 |
| **Slakh2100 (Baby)** | Real multi-instrument patterns → train L1 discriminator | Phase 2 |
| **MUSDB18** | Real audio spectrograms → fine-tune AST (L2 discriminator) | Phase 3 |

# Implementation Order

We followed a dependency-driven order: each step depends on the one before it.

1. **Datasets + Preprocessing**
   Download sounds, convert MIDI to grids, compute spectrograms
2. **Environment**
   Needs SampleLibrary from Step 1 to define action space
3. **Reward Functions**
   Plug into environment, test on hand-crafted grids
4. **Policy + Critic Networks**
   Input/output shapes defined by environment
5. **Train Phase 1 (Rules Only)**
   Sanity check — can the agent learn basic drums?
6. **Add Discriminator**
   Adversarial training with hybrid reward signal

**WE ARE HERE — Step 1 Complete**

# Dataset 1: Freesound Samples

The agent's sound library — actual WAV files placed on the beat grid

## How we sourced and processed

1. Connected to Freesound API via OAuth2 authentication
2. Searched by instrument tags (kick, snare, bass, etc.)
3. Downloaded top-rated candidates per category
4. Preprocessed: resample to 44100Hz, mono, normalize to -6dB
5. Trimmed/padded to fixed duration per category

## Current inventory

| Layer | Type | Count | Duration | Status |
|---|---|---|---|---|
| Layer 0 | Kick | 30 | 0.50s | PASS |
| Layer 1 | Snare | 30 | 0.50s | PASS |
| Layer 2 | Hi-hat | 25 | 0.50s | PASS |
| Layer 3 | Clap | 20 | 0.50s | PASS |
| Layer 4 | Bass | 30 | 1.00s | PASS |
| Layer 5 | Melody | 2 | 1.00s | LOW |
| Layer 6 | Pad | 1 | 2.00s | LOW |
| Layer 7 | FX | 1 | 1.00s | LOW |

Melody, Pad, FX need more samples — will fix before Phase 2. Not blocking Phase 1.

# Dataset 2: Groove MIDI

1,150 professional drum performances → 42,133 quantized beat grids

| 42,133 | 8 × 16 | 12.7% | 18 |
|---|---|---|---|
| Beat grids | Grid shape | Avg density | Music genres |

## Processing Pipeline

**Download** — MIDI-only zip from Google Magenta (3MB)

**Parse** — Read MIDI events using pretty_midi library

**Map** — Drum hits → 4 layers (kick, snare, hihat, clap)

**Quantize** — Snap timing to nearest 16th-note position

**Slice** — Cut performances into single-bar (16-step) segments

**Save** — Stack all grids → numpy array (42133, 8, 16)

Validation: Layers 4-7 all zeros (correct — drums only) | Kicks on strong beats confirmed | Grids visualized

# Dataset 3: BabySlakh

20-track subset of Slakh2100 — multi-instrument MIDI → 2,450 full 8-layer grids

| 2,450 | 99.6% | 20.2% | 20 |
|---|---|---|---|
| Beat grids | Have tonal content | Tonal density | Tracks (upgradeable) |

## Per-Layer Activity

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **Kick** | 14.2% | **Snare** | 10.2% | **Hi-hat** | 30.6% | **Clap** | 7.8% |
| **Bass** | 19.6% | **Melody** | 37.7% | **Pad** | 12.8% | **FX** | 10.5% |

## Key Difference from Groove MIDI

Groove MIDI has drums only (layers 4-7 empty). BabySlakh has all 8 layers populated — this is what the discriminator needs in Phase 2 to learn real multi-instrument arrangements. Can upgrade to full Slakh2100 (2,100 tracks) for ~200K+ grids.

# Dataset 4: MUSDB18

150 full-length professional songs → 31,323 mel-spectrogram patches for AST fine-tuning

| 31,323 | 128×130 | 4.2 GB | 150 |
|---|---|---|---|
| Spectrogram patches | Patch dimensions | Processed size | Full-length songs |

## Processing Pipeline

**Download** — Full MUSDB18 from Zenodo (4.7GB .stem.mp4)

**Decode** — Extract stems using stempeg (drums, bass, other)

**Resample** — 44100Hz → 22050Hz, stereo → mono

**Window** — Slice each stem into 3-second chunks

**Spectrogram** — 128-band mel-spectrogram per chunk

**Save** — Stack all patches → numpy array (31323, 128, 130)

Validation: Value range [-80, 0] dB | No NaN values | No silent patches | Spectrograms visualized

# Project File Structure

Everything stored on Google Drive — persistent across Colab sessions

```
datasets/
  ├── samples/
  kick/ (30)  snare/ (30)  hihat/ (25)  clap/ (20)
  bass/ (30)  melody/ (2)  pad/ (1)  fx/ (1)
  ├── samples_processed/
  Same 8 folders — standardized WAV files
  ├── groove_midi/
  Raw MIDI files from Google Magenta
  ├── groove_grids.npy
  42,133 drum grids — shape (42133, 8, 16)
  ├── babyslakh/
  20 tracks with MIDI + metadata
  ├── slakh_grids.npy
  2,450 multi-instrument grids — shape (2450, 8, 16)
  ├── musdb18/
  150 songs as .stem.mp4 files
  ├── musdb_spectrograms.npy
  31,323 patches — shape (31323, 128, 130)
```

## Storage Summary

| | |
|---|---|
| Freesound (raw) | ~500 MB |
| Freesound (processed) | ~200 MB |
| Groove MIDI (raw) | 3 MB |
| Groove grids (.npy) | ~5 MB |
| BabySlakh (raw) | ~1.4 GB |
| Slakh grids (.npy) | ~0.3 MB |
| MUSDB18 (raw) | ~4.7 GB |
| MUSDB spectrograms | ~4.2 GB |
| **TOTAL** | **~11 GB** |

# Phase Readiness

What's ready for each training phase

## Phase 1: Drums Only

**READY**

- ✓ Drum samples (105 files)
- ✓ Groove MIDI grids (42,133)
- ○ Environment
- ○ Rule-based rewards
- ○ PPO agent

## Phase 2: Full Instruments

**PARTIAL**

- ✓ Drum samples
- ✓ Slakh grids (2,450)
- ○ Melody/Pad/FX samples
- ○ L1 discriminator
- ○ Expanded action space

## Phase 3: Audio Effects

**DATA READY**

- ✓ MUSDB18 spectrograms (31,323)
- ○ AST model
- ○ SAC algorithm
- ○ Audio renderer
- ○ Effect parameter heads

# Next Steps

NOW

### Build BeatGridEnv

Gymnasium environment with 8×16 grid, MultiDiscrete action space, phase masking

NOW

### Implement reward functions

6 rule-based sub-rewards + compatibility scoring, tested on hand-crafted grids

NEXT

### Build policy/critic networks

CNN + embedding feature extractor, 3 action heads (layer, step, sample)

NEXT

### Train Phase 1 (rules only)

PPO agent learns drum patterns — our first sanity check

BEFORE P2

### Fix melody/pad/fx samples

Re-run Freesound API with relaxed queries for tonal categories

PHASE 2

### Build L1 discriminator

Custom transformer encoder trained on Groove MIDI + Slakh grids

**Dataset loading complete — ready to build the environment**